IMPLEMENTATION PLAN — PLAIN LANGUAGE

How the Resume Tailoring Feature Gets Built

A walkthrough of the five build phases, in the order they happen, what each one delivers, how it gets checked before moving on, and what stays exactly the same for you until you decide to turn something on. Companion to the full architecture document and the Claude Code implementation prompt.

The big picture, in one paragraph

Nothing about how the app behaves today changes the moment this work starts. Every new piece — the résumé tailoring, the smarter retry logic — is built with an on/off switch that defaults to **off**. The app keeps working exactly as it does now while the new pieces are built and tested underneath it. Only once you've reviewed what the new pieces actually produce do you flip a switch to let them affect a real application.

Why five phases, and why this order

Each phase produces something that can be checked on its own before the next phase depends on it. This matters because the riskiest part of this feature — an AI writing résumé content — sits at the end, built on top of foundations that are proven solid first. Building it in the reverse order (résumé generation first, plumbing later) would mean discovering foundational problems only after the risky part was already built on shaky ground.

The five phases

1 Give every job site the same "shape"

`no visible change`

Right now, the code that talks to Naukri is written a little differently from the code that talks to Wellfound, which is different again from Hirist — they all work, but they don't all follow the exact same blueprint. This phase writes that blueprint down formally (in programmer terms, an "interface") and makes all six sites follow it, without touching the actual clicking-and-typing logic that already works.

Think of it like standardizing plug shapes before you build a power strip — you're not changing what any of the individual devices do, you're making sure they'll all fit the same socket later.

How it's checked: run the app exactly as before, on every site, and confirm nothing behaves differently. This phase is only allowed to be invisible.

2 Teach the app to tell "needs a human" apart from "just try again"

`off by default`

Today, if something goes wrong applying to a job, the app just tries again a fixed number of times, no matter what went wrong. This phase teaches it to recognize the difference: a page that loaded slowly is worth retrying immediately; a security check that only you can answer (a CAPTCHA, a "confirm it's you" prompt, a code sent to your email) is never worth retrying blindly — doing that is exactly the kind of behavior that makes a website suspect it's talking to a bot, not a person.

Instead, for the "only a human can solve this" situations, the app checks a few things first: are you likely at your computer right now (based on when you last used it)? Is it a time of day you're usually around? Based on how quickly you've replied to similar requests before, is it worth a short wait right now, or better to send you a message and quietly try that job again later, once it hears back from you?

How it's checked: a set of test scenarios (CAPTCHA appeared, session expired, page hiccup, etc.) each verified to produce the right decision, without needing to actually trigger those situations live on a real job site.

3 Build the résumé tailoring itself — in "watch only" mode first

`off by default, then watch-only`

This is the heart of the feature, and it's built in stages of its own:

- **First**, for every job, check whether a résumé already tailored for a very similar job exists — most senior architecture/leadership postings ask for very similar things, so this reuse step handles the majority of cases for free, instantly, with no AI writing involved at all.
- **If nothing similar exists**, first try purely mechanical adjustments — reordering and re-emphasizing things that are already true on your résumé to better match this job's wording. Still no AI writing.
- **Only if that's not enough**, ask the local AI model already running on your machine to rewrite one small section at a time — never the whole résumé at once, and it's given a hard rule: never state anything not already documented in your résumé facts file. Every sentence it produces is then double-checked against those facts afterward — if it invented anything, that part gets removed, not left in.
- **Rarely**, if a job asks about something your facts file genuinely doesn't cover, you'll get at most two quick questions over Telegram — not a long form, and it won't sit there waiting forever if you don't answer.

Every tailored résumé is checked for length (exactly two pages) and format (no tables, columns, or graphics that confuse applicant-tracking software) before it's considered finished.

> "**Watch only" mode is the safety net here.** When this phase is first turned on, the app generates tailored résumés and shows you what it would have submitted — but keeps submitting your existing résumé for real, exactly as it does today. You get to review a batch of these before a second switch lets it actually start using them.

How it's checked: run a real (but safe, "practice mode") batch of job searches, look at a handful of the generated résumés by hand, and confirm nothing was invented and the formatting looks right.

4 Keep a simple running tally of what the AI is being asked to do

`small, built alongside phase 3`

Every time the app asks the local AI model to do something, that gets logged — what it was asked, roughly how long it took. Since everything runs on your own machine (never a paid cloud AI service), this isn't about a dollar bill — it's about being able to glance at a report and notice if something's asking the AI far more often than it should, before that becomes a real slowdown.

How it's checked: a simple command you can run that prints a summary — how many times, roughly how much time spent.

5 Wire up the on/off switches, write tests, update the instructions

`final polish`

All the new settings (tailoring on/off, watch-only on/off, the smarter retry logic on/off, and a few tuning numbers) get added to the same settings file style you already use, with everything defaulting to "off" or "safe." A written test suite is added so that future changes can be checked automatically rather than by re-running everything by hand. The project's own instructions get updated to explain the new settings in the same style as everything else already documented.

What "done" looks like

CHECK	WHAT IT CONFIRMS
Every phase individually verified	Nothing was rushed past without a check
App still runs fine with everything off	Zero risk to what already works, at any point
A full test run with everything on, in watch-only mode	The new pieces work end-to-end, without touching a real application yet
No calls to any paid AI service, anywhere	The $0-cost design goal is real, not just claimed
A written summary of what was built, what was skipped, and any judgment calls made	You know exactly what you're getting before you turn anything on

What you'll need to decide, once it's built

1. Review a batch of "watch only" tailored résumés — do they read well, are they accurate, do they look properly formatted?
2. Decide when to flip the switch that lets tailored résumés actually get submitted.
3. Decide when to turn on the smarter retry logic (it's independent of the résumé feature — you can turn on one without the other).

Companion to: resume-tailoring-sda.pdf (full architecture) and docs/design/IMPLEMENTATION_PROMPT.md (the technical build prompt for Claude Code)